

Negative-Weight Single-Source Shortest Paths in Near-Linear Time

By Aaron Bernstein, Danupon Nanongkai, and Christian Wulff-Nilsen

Abstract

In the single-source shortest paths problem, the goal is to compute the shortest path tree from a designated source vertex in a weighted, directed graph. We present the first near-linear-time algorithm for the problem that can also handle negative edge-weights; the runtime is $O(m \log^8(n) \log W)$.

In contrast to all recent developments that rely on sophisticated continuous optimization methods and dynamic algorithms, our algorithm is simple: It requires only a simple graph decomposition and elementary combinatorial tools. In fact, ours is the first combinatorial algorithm for negative-weight single-source shortest paths to break through the classic $\tilde{O}(m\sqrt{n} \log W)$ bound from more than three decades ago (Gabow and Tarjan, SICOMP'89.)

1. INTRODUCTION

We consider the single-source shortest paths (SSSP) problem with (possibly negative) integer weights. Given an m -edge n -vertex directed weighted graph $G = (V, E, w)$ with integral edge weight $w(e)$ for every edge $e \in E$ and a source vertex $s \in V$, the goal is to compute a shortest path from s .

Two textbook algorithms for SSSP are Bellman-Ford and Dijkstra's algorithm. Dijkstra's algorithm is near-linear time ($O(m + n \log n)$ time), but restricted to *nonnegative* edge weights. The Bellman-Ford algorithm^{5,17} can handle negative weights and only requires that there is no *negative-weight cycle* reachable from s in G ; in particular, the algorithm either returns a shortest path tree from s or returns a cycle reachable from s whose total weight is negative. Unfortunately, the runtime of Bellman-Ford is $O(mn)$.

Designing faster algorithms for SSSP with negative edge weights (denoted negative-weight SSSP) is one of the most fundamental and long-standing problems in graph algorithms. In the 1980s and 1990s, the scaling technique led to a wave of improvements over Bellman-Ford (Gabow,¹⁸ Gabow and Tarjan,¹⁹ and Goldberg²⁰), culminating in runtime $O(m\sqrt{n} \log W)$, where $W \geq 2$ is the minimum integer, such that $w(e) \geq -W$ for all $e \in E$. In the last few years, advances in continuous optimization and dynamic algorithms have led to a new wave of improvements, which achieve faster algorithms for the more general problems of transshipment and min-cost flow, and thus imply the same bounds for negative-weight SSSP (Cohen et al.,¹⁵ Axiotis et al.,³ BLNPSSSW,^{9,10,11} This line of work

resulted in a near-linear runtime ($\tilde{O}((m + n^{1.5}) \log W)$ time) on moderately dense graphs¹⁰ and $m^{4/3+o(1)} \log W$ runtime on sparse graphs.^{3,a} This state of the art motivates two natural questions:

1. *Can we get near-linear runtime for all graphs?*
2. *Can we achieve efficient algorithms without complex machinery?*

For the second question, note that currently all state-of-the-art results for negative-weight SSSP are based on min-cost flow algorithms, and hence rely on sophisticated continuous optimization methods and a number of complex dynamic algebraic and graph algorithms (for example, Bernstein et al.,⁶ Chuzhoy et al.,¹⁴ Nanongkai et al.,²⁴ and Saranurak and Wang²⁵). It would be useful to develop simple efficient algorithms that are specifically tailored to negative-weight SSSP, and thus circumvent the complexity currently inherent in flow algorithms; the best known bound of this kind is still the classic $O(m\sqrt{n} \log W)$ from more than three decades ago.^{19,20} A related question is whether it is possible to achieve efficient algorithms for the problem using combinatorial tools, or whether there are fundamental barriers that make continuous optimization necessary.

Our Result. In this paper we resolve both of the above questions for negative-weight SSSP: We present a simple combinatorial algorithm that reduces the running time all the way down to near-linear.

THEOREM 1.

There exists a randomized (Las Vegas) algorithm that takes $O(m \log^8(n) \log(W))$ time with high probability (and in expectation) for an m -edge input graph G_{in} and source s_{in} . It either returns a shortest path tree from s_{in} or returns a negative-weight cycle.

Independent result and follow-up work. Independently from our result, the recent major breakthrough by Chen et al.¹³ culminates the line of works based on continuous optimization and dynamic algorithms and achieves an almost-linear time bound of $O(m^{1+o(1)} \log^2 U)$ for minimum-cost flow.^b This directly implies a runtime of $m^{1+o(1)} \log(W)$ for negative-weight shortest paths.

In this paper, we prioritize simplicity and modularity, and not optimizing the logarithmic factors. The follow-up work by Bringmann, Cassis, and Fischer¹² greatly optimizes our framework to reduce runtime time to $O(m \log^2(n) \log(nW))$.

The original version of this paper was published in *Proceedings of the 63rd IEEE Symp. on Foundations of Computer Science* (2022).

a $\tilde{O}$ -notation hides polylogarithmic factors.

b The runtime is for when vertex demands, edge costs, and upper/lower edge capacities are all integral and bounded by U in absolute value.

) $\log \log n$). There is also follow-up work by Ashvinkumar *et al.* showing how the framework can be applied to the parallel and distributed models of computation.¹

2. PRELIMINARIES

Throughout, we only consider graphs with integer weights. For any weighted graph $G = (V, E, w)$, define $V(G) = V$, $E(G) = E$, and

$$E^{neg}(G) := \{e \in E \mid w(e) < 0\}.$$

Define $W_G := \max\{2, -\min_{e \in E} \{w(e)\}\}$; that is, W_G is the most negative edge weight in the graph.^c Given any set of edges $S \subseteq E$ we define $w(S) = \sum_{e \in S} w(e)$. We say that a cycle C in G is a negative-weight cycle if $w(C) < 0$. We define $\text{dist}_G(u, v)$ to be the shortest distance from u to v in graph G ; if there is a negative-weight cycle on some uv -path then we define $\text{dist}_G(u, v) = -\infty$.

Consider graph $G = (V, E, w)$ and consider subsets $V' \subseteq V$ and $E' \subseteq E$. We define $G[V']$ to be the subgraph of G induced by V' . We slightly abuse notation and denote a subgraph of G as $H = (V', E', w)$ where the weight function w is restricted to edges in H . We define $G \setminus V' = G[V \setminus V']$ and $G \setminus E' = (V, E \setminus E', w)$; that is, they are graphs where we remove vertices and edges in V' and E' respectively. We sometimes write $G \setminus v$ and $E \setminus e$ instead of $G \setminus \{v\}$ and $G \setminus \{e\}$, respectively, for any $v \in V$ and $e \in E$. We say that a subgraph H of G has *weak* diameter D if for any $u, v \in V(H)$ we have that $\text{dist}_G(u, v) \leq D$. We always let G_{in} and s_{in} refer to the main input graph/source of Theorem 1.

ASSUMPTION 1 (PROPERTIES OF INPUT GRAPH G_{in}).

We assume throughout the paper that the main input graph $G_{in} = (V, E, w_{in})$ satisfies the following properties:

1. $w_{in}(e) \geq -1$ for all $e \in E$ (thus, $W_{G_{in}} = 2$).
2. Every vertex in G_{in} has constant out-degree.

In the full version of the paper,⁷ we show that using existing techniques, one can easily reduce the general case to the more restricted case of this assumption, while only incurring a polylogarithmic overhead in the running time.

Dummy source and negative edges. The definitions below capture a common transformation we apply to negative weights and also allow us to formalize the number of negative edges on a shortest path. Note that most of our algorithms/definitions will not refer to the input source s_{in} , but instead to a dummy source that has edges of weight 0 to every vertex.

Definition 1.

Given any graph $G = (V, E, w)$, we let $G_s = (V \cup \{s\}, E \cup \{(s, v) \}_{v \in V}, w_s)$ refer to the graph G with a dummy source s added, where there is an edge of weight 0 from s to v for every $v \in V$ and no edges into s . Note that G_s has a negative-weight cycle if and only if G does and that $\text{dist}_{G_s}(s, v) = \min_{u \in V} \text{dist}_G(u, v)$.

For any integer B , let $G^B = (V, E, w^B)$ denote the graph obtained by adding B to all negative edge weights in G , that is, $w^B(e) = w(e) + B$ for all $e \in E^{neg}(G)$ and $w^B(e) = w(e)$ for $e \in E \setminus E^{neg}(G)$. Note that $(G^B)_s = (G_s)^B$ so we can simply write $G_s^B = (V \cup \{s\}, E \cup \{(s, v) \}_{v \in V}, w_s^B)$.

Definition 2.

For any graph $G = (V, E, w)$ and $s \in V$ such that s can reach all vertices in V , define $\eta_G(v) := \infty$ if $\text{dist}_G(s, v) = -\infty$; otherwise, define

$$\eta_G(v) := \min \{ |E^{neg}(G) \cap P| : P \text{ is a shortest } sv\text{-path in } G \}$$

Let $\eta(G; s) = \max_{v \in V} \eta_G(v; s)$. When $\text{dist}_G(s, v) \neq -\infty$, let $P_G(v; s)$ be a shortest sv -path on G such that

$$|E^{neg}(G) \cap P_G(v; s)| = \eta_G(v). \quad (1)$$

We omit s when s is the dummy source in G_s defined in Definition 1; that is, $\eta_G(v) = \eta_G(v; s)$, $\eta(G) = \eta(G_s; s)$, and $P_G(v) = P_G(v; s)$. Further, the graph G is omitted if the context is clear.

2.1. Price functions and equivalence.

Our algorithm heavily relies on price functions, originally introduced by Johnson²¹

Definition 3.

Consider a graph $G = (V, E, w)$ and let ϕ be any function: $V \rightarrow \mathbb{Z}$, where $\mathbb{Z}$ is the set of integers. Then, we define w_ϕ to be the weight function $w_\phi(u, v) = w(u, v) + \phi(u) - \phi(v)$ and we define $G_\phi = (V, E, w_\phi)$. We will refer to ϕ as a *price* function on V . Note that $(G_\phi)_\psi = G_{\phi+\psi}$.

Definition 4.

We say that two graphs $G = (V, E, w)$ and $G' = (V, E, w')$ are *equivalent* if (1) any shortest path in G is also a shortest path in G' and vice versa and (2) G contains a negative-weight cycle if and only if G' does.

LEMMA 1.²¹

Consider any graph $G = (V, E, w)$ and price function ϕ . For any pair $u, v \in V$ we have $\text{dist}_{G_\phi}(u, v) = \text{dist}_G(u, v) + \phi(u) - \phi(v)$, and for any cycle C we have $w(C) = w_\phi(C)$. As a result, G and G_ϕ are equivalent. Finally, if $G' = (V, E, w')$, $G' = (V, E, w')$ and $w' = c \cdot w(e)$ for some positive c , then G and G' are equivalent.

The overall goal of our algorithm will be to compute a price function ϕ such that all edge weights in G_ϕ are non-negative (assuming no negative-weight cycle); we can then run Dijkstra on G_ϕ . The lemma below, originally used by Johnson, will be one of the tools we use.

LEMMA 2.²¹

Let $G = (V, E)$ be a directed graph with no negative-weight cycle and let s be any vertex in G that can reach all vertices in V . Let $\phi(v) = \text{dist}_G(s, v)$ for all $v \in V$. Then, all edge weights in G_ϕ are non-negative. (The lemma follows trivially from the fact that $\text{dist}(s, v) \leq \text{dist}(s, u) + w(u, v)$.)

3. THE FRAMEWORK

In this section we describe the input/output guarantees of all the subroutines used in our algorithm, as well as some of the algorithms themselves.

3.1. Key subroutine: Low-diameter decomposition.

Low-diameter decomposition in *undirected* graphs has been

^c We set $W_G \geq 2$ so that we can write $\log(W_G)$ in our runtime.

a key tool in graph algorithms dating back to the 1980s, and there exist many variations of this core technique (for example, Awerbuch,² Bartal,⁴ Linial and Saks,²² and Miller et al.²³). More recent work presents a different formulation that is more appropriate to *directed* graphs,⁸ which is what we use in our paper.

In particular, one of our key subroutines is an algorithm that decomposes any directed graph with *non-negative* edge weights into strongly connected components (SCCs) of small diameter. In particular, the algorithm computes a small set of edges E^{rem} such that all SCCs in the graph $G \setminus E^{rem}$ have small weak diameter. Although the lemma below only applies to non-negative weights, we will show that it is in fact extremely useful for our problem.

LEMMA 3.

There is algorithm $\text{LowDiamDecomposition}(G, D)$ with the following guarantees:

- INPUT: an m -edge, n -vertex graph $G = (V, E, w)$ with non-negative integer edge weight function w and a positive integer D .
- OUTPUT: A set of edges E^{rem} with the following guarantees:
 - Each SCC of $G \setminus E^{rem}$ has weak diameter at most D ; that is, if u, v are in the same SCC, then $\text{dist}_G(u, v) \leq D$ and $\text{dist}_G(v, u) \leq D$.
 - For every $e \in E$, $\Pr[e \in E^{rem}] = O\left(\frac{w(e) \cdot \log^2 n}{D} + n^{-10}\right)$. These probabilities are not guaranteed to be independent.^d
- Running Time: The algorithm has running time $O(m \log^2 n + n \log^3 n)$.

This decomposition is new to this paper, but similar to other low-diameter decompositions used in both undirected and directed graphs, and especially to the algorithm PARTITION of Bernstein, Probst-Gutenberg, and Wulff-Nilsen.⁸ The main difference is that algorithm needed to work in a dynamic setting, and as a result their algorithm is too slow for our purposes.

Note that all the subroutines for low-diameter decomposition (including ours) only apply to graphs with *non-negative* edge weights. One of our main conceptual contributions is showing how to apply the subroutine to a problem with negative edge weights.

In this extended abstract, we use the above decomposition as a black box; details of the implementation can be found in the full version of the paper.⁷

3.2. Other subroutines.

LEMMA 4 (Dijkstra).

There exists an algorithm called $\text{Dijkstra}(G, s)$ that takes as input a graph G with non-negative edge weights and a vertex $s \in V$ and outputs a shortest path tree from s in G . The running time is $O(m + n \log(n))$.

It is easy to see that if G is a directed acyclic graph (DAG), computing a price function ϕ such that G_ϕ has non-negative edge weights is straightforward: simply loop over the topological order $v_1, \dots, v_n$ and set $\phi(v_i)$ so that all incoming edges have non-negative weight.

The lemma below generalizes this approach to graphs where only the “DAG part” has negative edges.

LEMMA 5 (FixDAGEdges).

There exists an algorithm $\text{FixDAGEdges}(G, \mathcal{P})$ that takes as input a graph G and a partition $\mathcal{P} := \{V_1, V_2, \dots\}$ of vertices of G such that

1. for every i , the induced subgraph $G[V_i]$ contains no negative-weight edges, and
2. when we contract every V_i into a node, the resulting graph is a DAG (that is, contains no cycle).

The algorithm outputs a price function $\phi: V \rightarrow \mathbb{Z}$ such that $w_\phi(u, v) \geq 0$ for all $(u, v) \in E$. The running time is $O(m + n)$.

PROOF SKETCH.

The algorithm is extremely simple: it loops over the SCCs V_i in topological order, and when it reaches V_i it sets the same price $\phi(v)$ for every $v \in V_i$ that ensures there are no non-negative edges entering V_i ; since all $\phi(v)$ are the same, this does not affect edge-weights inside V_i .

The next subroutine shows that computing shortest paths in a graph G can be done efficiently as long as $\eta(v)$ is small on average (see Definition 2 for $\eta(v)$). Note that this subroutine is the reason we use the assumption that every vertex has constant out-degree (Assumption 1).

LEMMA 6 (ElimNeg).

There exists an algorithm $\text{ElimNeg}(G, s)$ that takes as input a graph $G = (V, E, w)$ in which all vertices $v \neq s$ have constant out-degree and a source $s \in V$ that can reach all vertices in V . The algorithm outputs a price function ϕ such that $w_\phi(e) \geq 0$ for all $e \in E$ and has running time $O(\log(n) \cdot (n + \sum_{v \in V} \eta_G(v; s)))$ (Definition 2); note that if G contains a negative-weight cycle then $\sum_{v \in V} \eta_G(v; s) = \infty$ so the algorithm will never terminate and hence not produce any output.

PROOF SKETCH.

By Lemma 1, in order to compute the desired price function, it suffices to describe how $\text{ElimNeg}(G)$ computes $\text{dist}_G(s, v)$ for all $v \in V$, where s is the input source to $\text{ElimNeg}(G, s)$.

The algorithm is a straightforward combination of Dijkstra’s and Bellman-Ford’s algorithms. The algorithm maintains distance estimates $d(v)$ for each vertex v . It then proceeds in multiple iterations, where each iteration first runs a Dijkstra Phase that ensures that all non-negative edges are relaxed and then a Bellman-Ford Phase ensuring that all negative edges are relaxed. Consider a vertex v and let P be a shortest path from s to v in G with $\eta_G(v; s)$ edges of $E^{neg}(G)$. It is easy to see that $\eta_G(v; s) + 1$ iterations suffice to ensure that $d(v) = \text{dist}_G(s, v)$. In each of these iterations, v is

^d The 10 in the exponent suffices for our application but can be replaced by an arbitrarily large constant.

extracted from and added to the priority queue of the Dijkstra Phase only $O(1)$ times. Furthermore, after the $\eta_G(v; s) + 1$ iterations, v will not be involved in any priority queue operations. Since the bottleneck in the running time is the queue updates, we get the desired time bound of $O(\log(n) \cdot \sum_{v \in V} (\eta_G(v; s) + 1)) = O(\log(n) \cdot (n + \sum_{v \in V} \eta_G(v; s)))$.

3.3. The interface of the two main algorithms.

No negative-weight cycle assumption. Although it is possible to prove Theorem 1 directly, the need to return the negative-weight cycle somewhat complicates the details. For this reason, we have written both algorithms below to only return correct distances when the input graph contains no negative-weight cycle; if the graph does have a negative-weight cycle, the algorithms offer no guarantees, and in fact might not even terminate. So for the sake of the intuition, we recommend the reader to focus on the case when the graph G contains no negative-weight cycle. In the full version of the paper,⁷ we show that the weaker guarantees presented here can be transformed in a black-box manner to achieve the stronger guarantee of Theorem 1; the transformation incurs an extra $O(\log^2(n))$ factor in the runtime.

The two main algorithms. Our two main algorithms are called ScaleDown and SPmain. The latter is responsible for actually returning the shortest path tree and consists of a relatively simple outer shell. The main technical complexity lies in ScaleDown, which computes a price function that reduces the most negative edge weight by a factor of two. The procedure ScaleDown calls itself recursively, and the parameter δ tracks the layers of recursion.

THEOREM 2 (SPmain).

There exists an algorithm SPmain(G_{in}, s_{in}) that takes as input a graph G_{in} and a source s_{in} satisfying the properties of Assumption 1. If the algorithm terminates, it outputs a shortest path tree T from s_{in} . The running time guarantees are as follows:

- If the graph G_{in} contains a negative-weight cycle, then the algorithm never terminates.
- If the graph G_{in} does not contain a negative-weight cycle, then the algorithm has *expected* running time $\mathcal{T}_{spmain} = O(m \log^5(n))$.

THEOREM 3 (ScaleDown).

There exists the following algorithm ScaleDown($G = (V, E, w), \Delta, B$).

1. Input Requirements:
 - a. B is a positive integer, w is integral, and $w(e) \geq -2B$ for all $e \in E$
 - b. If the graph G does not contain a negative-weight cycle then the input must satisfy $\eta(G^B) \leq \Delta$; that is, for every $v \in V$ there is a shortest sv -path in G_s^B with at most Δ negative edges (Definitions 1, 2)
 - c. All vertices in G have constant out-degree
2. Output: If it terminates, the algorithm returns an integral price function ϕ such that $w_\phi(e) \geq -B$ for all $e \in E$
3. Running Time: If G does not contain a negative-weight cycle, then the algorithm has expected

runtime $O(m \log^3(n) \log(\Delta))$. Remark: If G contains a negative-weight cycle, there is no guarantee on the runtime, and the algorithm might not even terminate; but if the algorithm does terminate, it always produces a correct output.

4. ALGORITHM ScaleDown

We start by describing the algorithm ScaleDown, as this contains our main conceptual contributions; the much simpler algorithm SPmain is described in the following section. Full pseudocode of ScaleDown is given in Algorithm 1. The algorithm mostly works with graph $G^B = (V, E, w^B)$. For the analysis, the readers may want to familiarize themselves with, for example, G_s^B , w_s^B , $P_{G_s^B}(v)$, and $\eta(G^B)$ from Definitions 1 and 2. In particular, throughout this section, source s *always* refers to a dummy source that has outgoing edges to every vertex in V and has no incoming edges.

Note that $m = \Theta(n)$ since the input condition requires constant out-degree for every vertex. We thus use m and n interchangeably in this section. We briefly describe the ScaleDown algorithm and sketch the main ideas of the analysis in section 4.1; the full analysis follows in Section 4.2

Algorithm 1. Algorithm for ScaleDown($G = (V, E, w), \Delta, B$)

```

// Input/Output: See Theorem 3
1 if  $\Delta \leq 2$  then
2   Let  $\phi_2 = 0$  and jump to Phase 3 (Line 10)
3 Let  $d = \Delta/2$ . Let  $G_{\geq 0}^B := (V, E, w_{\geq 0}^B)$  where
    $w_{\geq 0}^B(e) := \max\{0, w^B(e)\}$  for all  $e \in E$ 
// Phase 0: Decompose  $V$  to SCCs  $V_1, V_2, \dots$ 
// with weak diameter  $dB$  in  $G$ 
4  $E^{rem} \leftarrow \text{LowDiamDecomposition}(G_{\geq 0}^B, dB)$ 
   (Lemma 3)
5 Compute Strongly Connected Components (SCCs) of
    $G^B \setminus E^{rem}$ , denoted by  $V_1, V_2, \dots$ 
// Properties: (Lemma 7) For each  $u, v \in V_i$ ,
    $\text{dist}_G(u, v) \leq dB$ .
// (Lemma 8) If  $\eta(G^B) \leq \Delta$ , then for every
    $v \in V_i$ ,  $E[P_{G^B}(v) \cap E^{rem}] = O(\log^2 n)$ 
// Phase 1: Make edges inside the SCCs  $G^B[V_i]$ 
// non-negative
6 Let  $H = \bigcup_i G[V_i]$ , i.e.  $H$  only contains edges inside
   the SCCs.
// (Lemma 9) If  $G$  has no negative-weight
   cycle, then  $\eta(H^B) \leq d = \Delta/2$ .
7  $\phi_1 \leftarrow \text{ScaleDown}(H, \Delta/2, B)$  // (Corollary 1)
    $w_{H\phi_1}^B(e) \geq 0$  for all  $e \in H$ 
// Phase 2: Make all edges in  $G^B \setminus E^{rem}$ 
// non-negative
8  $\psi \leftarrow \text{FixDAGEDges}(G_{\phi_1}^B \setminus E^{rem}, \{V_1, V_2, \dots\})$ 
   (Lemma 5)
9  $\phi_2 \leftarrow \phi_1 + \psi$  // (Lemma 10) All edges in
    $(G^B \setminus E^{rem})_{\phi_2}$  are non-negative
// Phase 3: Make all edges in  $G^B$ 
// non-negative
10  $\psi' \leftarrow \text{ElimNeg}((G_s^B)_{\phi_2}, s)$  (Lemma 6)
   // (Theorem 5) expected time  $O(m \log^3 m)$ .
   (To define  $(G_s^B)_{\phi_2}$  here, we define  $\phi_2(s) = 0$ .)
11  $\phi_3 = \phi_2 + \psi'$  // (Theorem 4) All edges in  $G_{\phi_3}^B$ 
   are non-negative.
12 return  $\phi_3$ ; // Since  $w_{\phi_3}^B(e) \geq 0$ , we have
    $w_{\phi_3}(e) \geq -B$ 

```

4.1. Overview.

The algorithm runs in phases, where in the last phase it calls $\text{ElimNeg}\left(\left(G_s^B\right)_{\phi_2}, s\right)$ for some price function ϕ_2 (Line 10). Recall (Lemma 6) that if ElimNeg terminates, it returns price function ψ' such that all edge weights in $\left(G_s^B\right)_{\phi_2+\psi'}$ are non-negative. Consequently, all edge weights in $G_{\phi_2+\psi'}^B$ are non-negative for $\phi_3 = \phi_2 + \psi'$ (since $G_{\phi_2}^B$ is a subgraph of $\left(G_s^B\right)_{\phi_2+\psi'}$). We thus have $w_{\phi_3}(e) \geq -B$ for all $e \in E$ as desired (because $w_{\phi_2}^B(e) \leq w(e) + B$). This already proves the output correctness of ScaleDown (Item 2 of Theorem 3). Thus it remains to bound the runtime when G contains no negative-weight cycle (Item 3). *In the rest of this subsection we assume that G contains no negative-weight cycle.*

Bounding the runtime when $\Delta \leq 2$ is easy: The algorithm simply jumps to Phase 3 with $\phi_2 = 0$ (Line 1 in Algorithm 1). Since $\eta(G^B) \leq \Delta \leq 2$ (the input requirement; Item 1b), the runtime of $\text{ElimNeg}\left(\left(G_s^B\right)_{\phi_2}, s\right)$ is $O\left((m + \sum_{v \in V} \eta_{G^B}(v)) \log m\right) = O(m\Delta \log m) = O(m \log m)$.

For $\Delta > 2$, we require some properties from Phases 0-2 in order to bound the runtime of Phase 3. In Phase 0, we partition vertices into (SCCs)^e $V_1, V_2, \dots$ such that each V_i has weak diameter $dB = B\Delta/2$ in G . We do this by calling

$$E^{\text{rem}} \leftarrow \text{LowDiamDecomposition}\left(G_{\geq 0}^B, dB\right),$$

where $G_{\geq 0}^B$ is obtained by rounding all negative weights in G^B up to 0; we then let $V_1, V_2, \dots$ be the SCCs of $G^B \setminus E^{\text{rem}}$. (We need $G_{\geq 0}^B$ since $\text{LowDiamDecomposition}$ cannot handle negative weights.)

The algorithm now proceeds in three phases. In Phase 1 it computes a price function ϕ_1 that makes the edges inside each SCC V_i non-negative; in Phase 2 it computes ϕ_2 such that the edges between SCCs in $G^B \setminus E^{\text{rem}}$ are also non-negative; finally, in Phase 3 it makes non-negative the edges in E^{rem} by calling ElimNeg .

(Phase 1) Our goal in Phase 1 is to compute ϕ_1 such that $w_{\phi_1}^B(e) \geq 0$ for every edge e in $G^B[V_i]$ for all i . To do this, we recursively call $\text{ScaleDown}(H, \Delta/2, B)$, where H is a union of all the SCCs $G[V_i]$. The main reason we can recursively call ScaleDown with parameter $\Delta/2$ is because we can argue that, when G does not contain a negative-weight cycle,

$$\eta(H^B) \leq d = \Delta/2.$$

As a rough sketch, the above bound holds because if any shortest path P from dummy source s in some $(G^B[V_i])_s$ contains more than d negative-weight edges, then it can be shown that $w(P) < -dB$; this is the step where we crucially rely on the difference between $w^B(P)$ and $w(P)$. Combining $w(P) < -dB$ with the fact that $G^B[V_i]$ has weak diameter at most dB implies that G contains a negative-weight cycle.

(Phase 2) Now that all edges in $G_{\phi_1}^B[V_i]$ are non-negative, we turn to the remaining edges in $G^B \setminus E^{\text{rem}}$. Since these remaining edges (that is, those not in the SCCs) form a DAG, we can simply call $\text{FixDAGEdges}\left(G_{\phi_1}^B \setminus E^{\text{rem}}, \{V_1, V_2, \dots\}\right)$ (Lemma 5) to get a price function ψ such that all edges in $\left(G_{\phi_1}^B \setminus E^{\text{rem}}\right)_{\psi} = G_{\phi_2}^B \setminus E^{\text{rem}}$ are non-negative.

(Phase 3) By the time we reach this phase, the only negative edges remaining in G^B are the ones in E^{rem} , that is, $E^{\text{neg}}(G_{\phi_2}^B) \subseteq E^{\text{rem}}$. We call $\text{ElimNeg}\left(\left(G_s^B\right)_{\phi_2}, s\right)$ to eliminate these negative edges. We are now ready to show that the runtime of Phase 3, which is $O\left(m + \sum_{v \in V} \eta_{(G^B)}(v; s) \log m\right)$ (Lemma 6), is $O(m \log^3 m)$ in expectation. We do so by proving that for any $v \in V$,

$$E\left[\eta_{(G^B)}(v; s)\right] = O(\log^2 m).$$

Details are in the next section, but a competitive reader might want to prove this equation via a series of inequalities: $\eta_{(G^B)}(v) \leq |P_{G^B}(v) \cap E^{\text{neg}}(G_{\phi_2}^B)| + 1 \leq |P_{G^B}(v) \cap E^{\text{rem}}| + 1$, and also, the guarantees of $\text{LowDiamDecomposition}$ (Lemma 3) imply that after Phase 0, $E[|P_{G^B}(v) \cap E^{\text{rem}}|] = O(\log^2 m)$.

Finally, observe that there are $O(\log \Delta)$ recursive calls, and the runtime of each call is dominated by the $O(m \log^3 m)$ time of Phase 3. So, total expected runtime is $O(m \log^3(m) \log \Delta)$.

4.2. Full analysis.

Theorem 3 follows from Theorems 4 and 5. We start with Theorem 4, which is quite trivial to prove.

THEOREM 4.

$\text{ScaleDown}(G = (V, E, w), \Delta, B)$ either does not terminate or returns $\phi = \phi_3$ such that $w_{\phi}(e) \geq -B$ for all $e \in E$.

PROOF.

Consider when we call $\text{ElimNeg}\left(\left(G_s^B\right)_{\phi_2}, s\right)$ (Lemma 6) in Phase 3 for some integral price function ϕ_2 . Either this step does not terminate or returns an integral price function ψ' such that $\left(G_{\phi_2}^B\right)_{\psi'} = G_{\phi_2+\psi'}^B = G_{\phi_3}^B$ contains no negative-weight edges; that is, $w_{\phi_3}^B(e) \geq 0$ for all $e \in E$. Since $w^B(e) \leq w(e) + B$, we have $w_{\phi_3}(e) \geq w_{\phi_3}^B(e) - B \geq -B$ for all $e \in E$.

Theorem 4 implies that the output condition of ScaleDown (item 2 in Theorem 3) is always satisfied, regardless of whether G contains a negative-weight cycle or not. It remains to show that if G does not contain a negative-weight cycle, then $\text{ScaleDown}(G = (V, E, w), \Delta, B)$ has expected runtime of $O(m \log^3(m) \log(\Delta))$. It suffices to show the following.

THEOREM 5.

If G does not contain a negative-weight cycle, then the expected time complexity of Phase 3 is $O(m \log^3 m)$.

This suffices because, first of all, it is easy to see that Phase 0 requires $O(m \log^3(m))$ time (by Lemma 3) and other phases (except the recursion on Line 7) require $O(m + n)$ time. Moreover, observe that if G contains no negative-weight cycle, then the same holds for H in the recursion call $\text{ScaleDown}(H, \Delta/2, B)$ (Line 7 of Algorithm 1); thus, if G contains no negative-weight cycle, then all recursive calls also get an input with no negative-weight cycle. So, by Theorem 5 the time to execute a single call in the recursion tree is $O(m \log^3 m)$ in expectation. Since there are $O(\log \Delta)$ recursive calls, the total running time is $O(m \log^3(m) \log(\Delta))$ by linearity of expectation.

Proof of Theorem 5. The rest of this subsection is devoted to proving Theorem 5. From now on, we consider any graph G that does not contain a negative-weight cycle. (We often continue to state this assumption in lemma state-

^e Recall that a SCC is a *maximal* set $C \subseteq V$ such that for every $u, v \in V$, there are paths from u to v and from v to u . See, for example, Cormen et al.¹⁶

^f One can also use $G_{\geq 0}$ instead of $G_{\geq 0}^B$. We choose $G_{\geq 0}^B$ since some proofs become slightly simpler.

ments so they are self-contained.)

Base case: $\Delta \leq 2$. This means that for every vertex v , $\eta_{G^s}(v) \leq \eta(G^B) \leq \Delta \leq 2$ (see the input requirement of ScaleDown in Item 1b of Theorem 3). So, the runtime of Phase 3 is

$$O\left(m + \sum_{v \in V} \eta_{G^s}(v) \log m\right) = O(m\Delta \log m) = O(m \log m).$$

We now consider when $\Delta > 2$ and show properties achieved in each phase. We will use these properties from earlier phases in analyzing the runtime of Phase 3.

Phase 0: Low-diameter Decomposition. It is straightforward that the SCCs $G[V_i]$ have weak diameter at most dB (this property will be used in Phase 1):

LEMMA 7.

For every i and every $u, v \in V_i$, $\text{dist}_G(u, v) \leq dB$.

PROOF.

For every $u, v \in V_i$, we have

$$\text{dist}_G(u, v) \leq \text{dist}_{G_{\geq 0}^s}(u, v) \leq dB$$

where the first inequality is because $w(e) \leq w_{\geq 0}^B(e)$ for every edge $e \in E$ and the second inequality is by the output guarantee of LowDiamDecomposition (Lemma 3).

Another crucial property from the decomposition is this: Recall from Definition 2 that $P_{G^s}(v)$ is the shortest sv -path in G_s^B with $\eta_{G^s}(v)$ negative-weight edges. We show below that in expectation $P_{G^s}(v)$ contains only $O(\log^2 n)$ edges from E^{rem} . This will be used in Phase 3.

LEMMA 8.

If $\eta(G^B) \leq \Delta$, then for every $v \in V$,

$$E[|P_{G^s}(v) \cap E^{\text{rem}}|] = O(\log^2 m).$$

PROOF.

Consider any $v \in V$. The crux of the proof is the following bound on the weight of $P_{G^s}(v)$ in $G_{\geq 0}^B$:

$$w_{\geq 0}^B(P_{G^s}(v)) \leq \eta_{G^s}(v) \cdot B \quad (2)$$

where we define $w_{\geq 0}^B(s, u) = 0$ for every $u \in V$. Recall the definition of w_s^B from Definition 1 and note that $w_s^B(P_{G^s}(v)) \leq 0$ because there is an edge of weight 0 from s to every $v \in V$. We thus have Equation (2) because:

$$\begin{aligned} w_{\geq 0}^B(P_{G^s}(v)) &\leq w_s^B(P_{G^s}(v)) + |P_{G^s}(v) \cap E^{\text{neg}}(G^B)| \cdot B \\ &\leq |P_{G^s}(v) \cap E^{\text{neg}}(G^B)| \cdot B \\ &= \eta_{G^s}(v) \cdot B \end{aligned} \quad (3)$$

(The first inequality holds because $w^B(e) \geq -B$ for all $e \in E$; the second inequality holds because $w_s^B(P_{G^s}(v)) \leq 0$; the last inequality follows from the Definition of $P_{G^s}(v)$.)

Recall from the output guarantee of LowDiamDecomposition (Lemma 3) that $\Pr[e \in E^{\text{rem}}] = O\left(\frac{w_{\geq 0}^B(e)}{(\log n)^2/D + n^{-10}}\right)$, where in our case $D = dB = B\Delta/2$. This, the linearity of expectation, and (2) imply that

$$\begin{aligned} E[|P_{G^s}(v) \cap E^{\text{rem}}|] &= O\left(\frac{w_{\geq 0}^B(P_{G^s}(v)) \cdot (\log n)^2}{B\Delta/2} + |P_{G^s}(v)| \cdot n^{-10}\right) \\ &\stackrel{(2)}{=} O\left(\frac{2\eta_{G^s}(v) \cdot (\log n)^2}{\Delta} + n^{-9}\right) \end{aligned}$$

which is $O(\log^2 n)$ when $\eta(G^B) \leq \Delta$.

Phase 1: Make edges inside the SCCs $G^B[V_i]$ non-negative.

We argue that ScaleDown($H, \Delta/2, B$) is called with an input that satisfies its input requirements (Theorem 3). The most important requirement is $\eta(H^B) \leq \Delta/2$ (Item 1b) which we prove below (other requirements are trivially satisfied). Recall that we set $d := \Delta/2$ in Line 3.

LEMMA 9.

If G has no negative-weight cycle, then $\eta(H^B) \leq d = \Delta/2$.

PROOF.

Consider any vertex $v \in V$. Let $P := P_{H^s}(v) \setminus s$; that is, P is obtained by removing s from a shortest sv -path in H_s^B that contains $\eta_{H^s}(v)$ negative weights in H_s^B . Let u be the first vertex in P . Note three easy facts:

- a. $w_{H^s}(e) = w_H(e) + B$ for all $e \in E^{\text{neg}}(H^B)$,
- b. $|E^{\text{neg}}(H^B) \cap P| = |E^{\text{neg}}(H^B) \cap P_{H^s}(v)| = \eta_{H^s}(v)$, and
- c. $w_{H^s}(P) = w_{H^s}(P_{H^s}(v)) \leq w_{H^s}(s, v) = 0$,

where (b) and (c) are because the edges from s to u and v in H_s^B have weight zero. Then,^g

$$\begin{aligned} \text{dist}_G(u, v) &\leq w_H(P) \stackrel{(a)}{\leq} w_{H^s}(P) - |E^{\text{neg}}(H^B) \cap P| \cdot B \\ &\stackrel{(b)}{=} w_{H^s}(P) - \eta_{H^s}(v) \cdot B \stackrel{(c)}{\leq} -\eta_{H^s}(v) \cdot B. \end{aligned} \quad (4)$$

Note that u and v are in the same SCC V_i^h , thus, by Lemma 7:

$$\text{dist}_G(v, u) \leq dB. \quad (5)$$

If G contains no negative-weight cycle, then $\text{dist}_G(u, v) + \text{dist}_G(v, u) \geq 0$ and thus $\eta_{H^s}(v) \leq dB \cdot (1/B) = d$ by Equations (4) and (5). Since this holds for every $v \in V$, Lemma 9 follows. Consequently, ScaleDown (Theorem 3) is guaranteed to output ϕ_1 as follows.

COROLLARY 1.

If G has no negative-weight cycle, then all edges in $G_{\phi_1}^B[V_i]$ are non-negative for every i .

Phase 2: Make all edges in $G^B \setminus E^{\text{rem}}$ non-negative. Now that all edges in $G_{\phi_1}^B[V_i]$ are non-negative, we turn to the remaining edges in $G^B \setminus E^{\text{rem}}$. Intuitively, since these remaining edges (that is, those not in the SCCs) form a directed acyclic graph (DAG), calling FixDAGEdges($G_{\phi_1}^B \setminus E^{\text{rem}}, \{V_1, V_2, \dots\}$) (Lemma 5) in Phase 2 produces the following result.

LEMMA 10.

If G has no negative-weight cycle, all weights in $G_{\phi_2}^B \setminus E^{\text{rem}}$ are non-negative.

PROOF.

Clearly, $G_{\phi_1}^B \setminus E^{\text{rem}}$ and $\{V_1, V_2, \dots\}$ satisfy the input conditions of Lemma 5, that is, (1) $(G_{\phi_1}^B \setminus E^{\text{rem}})_{\phi_1}[V_i]$ contains no negative-weight edges for every i (this is due to Corollary 1), and (2) when we contract every V_i into a node, the resulting graph

g The " $\leq$ " part in (4) is not equality because there can be an edge in $E^{\text{neg}}(H) \cap P$ that is not in $E^{\text{neg}}(H^B) \cap P$.

h in fact all vertices in P are in the same SCC V_i , because we define $H = \bigcup_i G[V_i]$.

is a DAG (since the V_i are precisely the (maximal) SCCs of $G^B \setminus E^{rem}$).ⁱ Thus, $\text{FixDAGEdges}\left(\left(G^B \setminus E^{rem}\right)_{\phi_1}, \{V_1, V_2, \dots\}\right)$ returns ψ such that $\left(G^B \setminus E^{rem}\right)_{\phi_1}^\psi = G^B \setminus E^{rem}_{\phi_2}$ contains no negative-weight edges.

Phase 3 runtime. Now we are ready to prove Theorem 5, that is, the runtime bound of $\text{ElimNeg}\left(\left(G^B\right)_{\phi_2}, s\right)$ in Phase 3 when G contains no negative-weight cycle. We start by clarifying the nature of the graph $\left(G^B\right)_{\phi_2}$. We start with the graph G^B ; we then get G_s^B by adding a dummy source with outgoing edges of weight 0 to every $v \in V$; finally we get $\left(G_s^B\right)_{\phi_2}$ by applying price function ϕ_2 to G_s^B , where we define $\phi_2(s) = 0$. The reason we apply ϕ_2 at the end is to ensure that G_s^B and $\left(G_s^B\right)_{\phi_2}$ are equivalent. Note that after we apply ϕ_2 , the edges incident to s can have non-zero weight (positive or negative); but s can still reach every vertex, so we can still call ElimNeg with s as the source.

Recall (Lemma 6 and definition 2) that the runtime of $\text{ElimNeg}\left(\left(G_s^B\right)_{\phi_2}, s\right)$ is

$$O\left(\left(m + \sum_{v \in V} \eta_{(G_s^B)_{\phi_2}}(v; s)\right) \log m\right)$$

Fix any $v \in V$, observe that

$$\begin{aligned} \eta_{(G_s^B)_{\phi_2}}(v; s) &= \min\left\{\left|P \cap E^{neg}\left(\left(G_s^B\right)_{\phi_2}\right)\right| : P \text{ is a shortest } sv\text{-path in } \left(G_s^B\right)_{\phi_2}\right\} \\ &\leq |P_{G_s^B}(v) \cap E^{neg}\left(\left(G_s^B\right)_{\phi_2}\right)| \end{aligned}$$

where the inequality is because, for any price function ϕ_2 , $P_{G_s^B}(v)$ is a shortest sv -path in $\left(G_s^B\right)_{\phi_2}$ (because $\left(G_s^B\right)_{\phi_2}$ and G_s^B are equivalent and $P_{G_s^B}(v)$ is a shortest sv -path in G_s^B by definition). By Lemma 10, all negative-weight edges in G_s^B are in E^{rem} , that is, $E^{neg}\left(G_s^B\right) \subseteq E^{rem}$, so,

$$\eta_{(G_s^B)_{\phi_2}}(v; s) \leq |P_{G_s^B}(v) \cap E^{rem}| + 1$$

where the “+1” term is because the edge incident to s in $P_{G_s^B}(v)$ can also be negative. By Lemma 8 and the fact that $\eta(G^B) \leq \Delta$ (input requirement Item 1b in Theorem 3),^j

$$E\left[\eta_{(G_s^B)_{\phi_2}}(v; s)\right] \leq E\left[|P_{G_s^B}(v) \cap E^{rem}| + 1\right] = O(\log^2 m). \quad (6)$$

(The last equality is by Lemma 8.) Thus, the expected runtime of $\text{ElimNeg}\left(\left(G_s^B\right)_{\phi_2}, s\right)$ is

$$O\left(\left(m + E\left[\sum_{v \in V} \eta_{(G_s^B)_{\phi_2}}(v; s)\right]\right) \log m\right) = O(m \log^3 m).$$

5. ALGORITHM SPmain

In this section we present algorithm $\text{SPmain}(G_{in}, s_{in})$ (Theorem 2), which always runs on the main input graph G_{in} and source s_{in} .

Description of Algorithm SPmain(G_{in}, s_{in}). See Algorithm 2 for pseudocode. Recall that if G_{in} contains a negative-weight cycle, then the algorithm is not supposed to terminate; for intuition, we recommend the reader focus on the case where G_{in} contains no negative-weight cycle.

ⁱ See, for example, Lemma 22.13 in Cormen et al.¹⁶

^j The expectation in (6) is over the random outcomes of the low-diameter decomposition in Phase 0 and the recursion in Phase 1. Note that both $\eta_{(G_s^B)_{\phi_2}}(v; s)$ and $|P_{G_s^B}(v) \cap E^{rem}|$ are random variables. Since we always have $\eta_{(G_s^B)_{\phi_2}}(v; s) \leq |P_{G_s^B}(v) \cap E^{rem}| + 1$, we also have $E[\eta_{(G_s^B)_{\phi_2}}(v; s)] \leq E[|P_{G_s^B}(v) \cap E^{rem}|] + 1$.

The algorithm first creates an equivalent graph $\bar{G}$ by scaling up edge weights by $2n$ (Line 1), and also rounds B (Line 2), all to ensure that everything remains integral. It then repeatedly calls ScaleDown until we have a price function ϕ_t such that $w_{\phi_t}(e) \geq -1$ (See for loop in Line 4). The algorithm then defines a graph $G^* = (V, E, w^*)$ with $w^*(e) = w_{\phi_t}(e) + 1$ (Line 7). We will argue that because we are dealing with the scaled graph $\bar{G}$, the additive +1 is insignificant and does not affect the shortest path structure, so running Dijkstra on G^* will return correct shortest paths in G (Lines 8 and 9).

Correctness. We focus on the case where the algorithm terminates, and hence every line is executed. First we argue that weights in G^* (Line 7) are non-negative.

CLAIM 1.

If the algorithm terminates, then for all $e \in E$ and $i \in [0, t := \log_2(B)]$ we have that $\bar{w}_i$ is integral and that $\bar{w}_i(e) \geq -B/2^i$ for all $e \in E$. Note that this implies that $\bar{w}_i(e) \geq -1$ for all $e \in E$, and so the graph G^* has non-negative weights.

PROOF.

We prove the claim by induction on i . For base case $i = 0$, the claim holds for $\bar{G}_{\phi_0} = \bar{G}$ because $w_{in}(e) \geq -1$ (see Assumption 1), so $\bar{w}(e) \geq -2n \geq -B$ (see Lines 1 and 2).

Now assume by induction that the claim holds for $\bar{G}_{\phi_{i-1}}$. The call to $\text{ScaleDown}(\bar{G}_{\phi_{i-1}}, \Delta := n, B/2^i)$ in Line 5 satisfies the necessary input properties (See Theorem 3): property 1a holds by the induction hypotheses; property 1b holds because we have $\eta_G(v) \leq n$ for any graph G with no negative-weight cycle; property 1b holds because G_{in} has constant out-degree, and the algorithm never changes the graph topology. Thus, by the output guarantee of ScaleDown we have that $\left(\bar{w}_{\phi_{i-1}}\right)_{\psi_i}(e) \geq (B/2^{i-1})/2 = B/2^i$. The claim follows because as noted in Definition 3,

$$\left(\bar{w}_{\phi_{i-1}}\right)_{\psi_i} = \bar{w}_{\phi_{i-1} + \psi_i} = \bar{w}_{\phi_i}.$$

COROLLARY 2.

If G_{in} contains a negative-weight cycle then the algorithm does not terminate.

Algorithm 2. SPmain($G_{in} = (V, E, w_{in}), s_{in}$)

```

1  $\bar{w}(e) \leftarrow w_{in}(e) \cdot 2n$  for all  $e \in E$ ,  $\bar{G} \leftarrow (V, E, \bar{w})$ ,  
    $B \leftarrow 2n$ . // scale up edge weights
2 Round  $B$  up to nearest power of 2 // still have  
    $\bar{w}(e) \geq -B$  for all  $e \in E$ 
3  $\phi_0(v) = 0$  for all  $v \in V$  // identity price function
4 for  $i = 1$  to  $t := \log_2(B)$  do
5    $\psi_i \leftarrow \text{ScaleDown}(\bar{G}_{\phi_{i-1}}, \Delta := n, B/2^i)$ 
6    $\phi_i \leftarrow \phi_{i-1} + \psi_i$  // (Claim 1)  $w_{\phi_i}(e) \geq -B/2^i$   
   for all  $e \in E$ 
7  $G^* \leftarrow (V, E, w^*)$  where  $w^*(e) \leftarrow \bar{w}_{\phi_t}(e) + 1$  for all  
    $e \in E$ .  
   // Observe:  $G^*$  in above line has  
   non-negative weights
8 Compute a shortest path tree  $T$  from  $s$  using  
   Dijkstra( $G^*, s$ ) (Lemma 4)
// (Claim 2) Will Show: any shortest path in  
    $G^*$  is also shortest in  $G$ 
9 return shortest path tree  $T$ .
```

PROOF.

Say, for contradiction, that the algorithm terminates; then by Lemma 1 we have that $\bar{w}_{\phi_i}(e) \geq -1$. Now, let C be any negative-weight cycle in G_{in} . Since all weights in $\bar{G}$ are multiples of $2n$ (Line 1), we know that $\bar{w}(C) \leq -2n$. But we also know that $\bar{w}_{\phi_i}(C) \geq -|C| \geq -n$. So $\bar{w}(C) \neq \bar{w}_{\phi_i}(C)$, which contradicts Lemma 1.

Now we show that the algorithm produces a correct output.

CLAIM 2.

Say that G_{in} contains no negative-weight cycle. Then, the algorithm terminates, and if P is a shortest sv-path in G^* (Line 7) then it is also a shortest sv-path in G_{in} . Thus, the shortest path tree T of G^* computed in Line 9 is also a shortest path tree in G_{in} .

PROOF.

The algorithm terminates because each call to ScaleDown($\bar{G}_{\phi_{i+1}}, \Delta := n, B/2^i$) terminates, because $\bar{G}_{\phi_{i+1}}$ is equivalent to G_{in} (Lemma 1) and so does not contain a negative-weight cycle. Now we show that

$$P \text{ is also a shortest path in } \bar{G}_{\phi_i} \quad (7)$$

which implies the claim because $\bar{G}_{\phi_i}$ and G_{in} are equivalent. We assume that $s \neq v$ because otherwise the claim is trivial. Observe that since all weights in $\bar{G}$ are multiples of $2n$ (Line 1), all shortest distances are also multiples of $2n$, so for any two sv-paths P_{sv} and P'_{sv} , $|\bar{w}(P_{sv}) - \bar{w}(P'_{sv})|$ is either 0 or $> n$. It is easy to check that by Lemma 1, we also have that

$$|\bar{w}_{\phi_i}(P_{sv}) - \bar{w}_{\phi_i}(P'_{sv})| \text{ is either 0 or } > n. \quad (8)$$

Moreover, by definition of G^* we have

$$\bar{w}_{\phi_i}(P_{sv}) < w^*(P_{sv}) = \bar{w}_{\phi_i}(P_{sv}) + |P_{sv}| < \bar{w}_{\phi_i}(P_{sv}) + n. \quad (9)$$

Now, we prove (7). Assume for contradiction that there was a shorter path P' in $\bar{G}_{\phi_i}$. Then,

$$\bar{w}_{\phi_i}(P) - \bar{w}_{\phi_i}(P') > n. \quad (10)$$

So, $w^*(P) < \bar{w}_{\phi_i}(P) + n < \bar{w}_{\phi_i}(P') + n < w^*(P')$, contradicting P being shortest in G^* .

Running Time Analysis. By Corollary 2, if G_{in} does not contain a negative-weight cycle then the algorithm does not terminate, as desired. We now focus on the case where G_{in} does not contain a negative-weight cycle. The running time of the algorithm is dominated by the $\log(B) = O(\log(n))$ calls to ScaleDown($\bar{G}_{\phi_{i+1}}, \Delta := n, B/2^i$). Note that all the input graphs $\bar{G}_{\phi_{i+1}}$ are equivalent to G_{in} , so they do not contain a negative-weight cycle. By Theorem 3, the expected runtime of each call to ScaleDown is $O(m \log^3(n) \log(\Delta)) = O(m \log^4(n))$. So, the expected runtime of SPmain is $O(m \log^5(n))$.

Acknowledgment

We thank Sepehr Assadi, Joakim Blikstad, Thatchaphol Saranurak, and Satish Rao for their comments on the early draft of the paper and the discussions. We thank Mohsen Ghaffari, Merav Parter, Satish Rao, and Goran Zuzic for pointing out

some literature on low-diameter decomposition. A discussion with Mohsen Ghaffari led to some simplifications of our low-diameter decomposition algorithm. We also thank Vikrant Ashvinkumar, Nairen Cao, Christoph Grunau, and Yonggang Jiang for pointing out a technical error in an earlier version of the paper. □

References

1. Ashvinkumar, V. et al. *Parallel and Distributed Exact Single-Source Shortest Paths with Negative Edge Weights*, 2023.
2. Awerbuch, B. Complexity of network synchronization. *J. ACM* 32, 4 (1985), 804–823. Announced at STOC'84.
3. Axiotis, K., Madry, A., and Vladu, A. Circulation control for faster minimum cost flow in unit-capacity graphs. In *FOCS*. IEEE, 2020, 93–104.
4. Bartal, Y. Probabilistic approximations of metric spaces and its algorithmic applications. In *37th Annual Symp. On Foundations of Computer Science, FOCS '96*, Burlington, Vermont, USA, 14–16 October, 1996. IEEE Computer Society, 1996, 184–193.
5. Bellman, R. On a routing problem. *Quarterly of Applied Mathematics* 16, 1, (1958), 87–90.
6. Bernstein, A. et al. Fully-dynamic graph sparsifiers against an adaptive adversary. *CoRR*, abs/2004.08432, 2020.
7. Bernstein, A., Nanongkai, D., and Wulff-Nilsen, C. Negative-weight single-source shortest paths in almost-linear time. *CoRR*, abs/2203.03456, 2022.
8. Bernstein, A., Probst Gutenberg, M., and Wulff-Nilsen, C. Near-optimal decremental SSSP in dense weighted digraphs. *61st IEEE Annual Symp. On Foundations of Computer Science, FOCS 2020*, Durham, NC, USA, November 16–19, 2020. S. Irani (ed.). IEEE, 2020, 1112–1122.
9. Brand, J.v.d. et al. Minimum cost flows, mdps, and l-regression in nearly linear time for dense instances. In *STOC*. ACM, 2021, 859–869.
10. Brand, J.v.d. et al. Bipartite matching in nearly-linear time on moderately dense graphs. In *FOCS*. IEEE, 2020, 919–930.
11. Brand, J.v.d., Lee, Y.T., Sidford, A., and Song, Z. Solving tall dense linear programs in nearly linear time. In *STOC*, 2020.
12. Bringmann, K., Cassis, A., and Fischer, N. Negative-weight single-source shortest paths in near-linear time: Now faster! *CoRR*, abs/2304.05279, 2023.
13. Chen, L. et al. *Maximum Flow and Minimum-Cost Flow in Almost-Linear Time*, Mar. 2022.
14. Chuzhoy, J. et al. A deterministic algorithm for balanced cut with applications to dynamic connectivity, flows, and beyond. In *FOCS*, 2020.
15. Cohen, M.B., Madry, A., Sankowski, P., and Vladu, A. Negative-weight shortest paths and unit capacity minimum cost flow in $O(m^{10/7} \log W)$ time. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symp. On Discrete Algorithms (SODA)*. SIAM, 2017, 752–771.
16. Cormen, T.H., Leiserson, C.E., Rivest, R.L., and Stein, C. *Introduction to Algorithms*, 3rd Edition. MIT Press, 2009.
17. Ford, R. *Paper P-923*. The RAND Corporation, Santa Monica, California, 1956.
18. Gabow, H.N. Scaling algorithms for network problems. *J. Comput. Syst. Sci.* 31 2 (1985), 148–168. announced at FOCS'83.
19. Gabow, H.N. and Tarjan, R.E. Faster scaling algorithms for network problems. *SIAM J. Comput.* 18 5 (1989), 1013–1036.
20. Goldberg, A.V. Scaling algorithms for the shortest paths problem. *SIAM J. Comput.* 24 3 (1995), 494–504. Announced at SODA'93.
21. Johnson, D.B. Efficient algorithms for shortest paths in sparse networks. *J. AC* 24 1 (1977), 1–13.
22. Linial, N. and Saks, M.E. Low diameter graph decompositions. *Comb* 13 4 (1993), 441–454.
23. Miller, G.L., Peng, R., and Xu, S.C. Parallel graph decompositions using random shifts. In *25th ACM Symp. on Parallelism in Algorithms and Architectures, SPAA '13*, Montreal, QC, Canada - July 23 - 25, 2013. G. E. Blelloch and B. Vöcking (eds.). ACM, 2013, 196–203.
24. Nanongkai, D., Saranurak, T., and Wulff-Nilsen, C. Dynamic minimum spanning forest with subpolynomial worst-case update time. In *FOCS*. IEEE Computer Society, 2017, 950–961.
25. Saranurak, T. and Wang, D. Expander decomposition and pruning: Faster, stronger, and simpler. In *SODA*. SIAM, 2019, 2616–2635.

Aaron Bernstein (burnstei@gmail.com), Rutgers University, New Brunswick, NJ, USA. This work was funded by Google Research Grant and NSF CAREER Grant #1942010.

Danupon Nanongkai (danupon@gmail.com), Max Planck Institute for Informatics, Saarbrücken, Germany. This project received funding from the European Research Council under the European Union's Horizon 2020 research

and innovation program under grant agreement No 715672. Partial funding was provided by the Swedish Research Council (Reg. No. 2019-05622). Work was done while at the University of Copenhagen.

Christian Wulff-Nilsen (koolooz@di.ku.dk), University of Copenhagen, Denmark. Work was supported by Starting Grant 7027-00050B from the Independent Research Fund Denmark under the Sapere Aude research career program.